



Do you know where your variables are?

PerlMonks

Re: find all paths of length n in a graph

by [blokhead](#)

on Jan 10, 2006 at 18:29 UTC (#522270=note: [print w/ replies](#), [xml](#))

in reply to [find all paths of length n in a graph](#)

Adjacency matrices are really nice for this problem since they make "paths of length N" problems easy to reason about. Probably just wandering through the graph and keeping track of paths will give simpler code, but I guess I prefer using heavy machinery when it has nice abstraction properties... I also happen to really how you can use the structure of the matrix multiplication algorithm to do a lot of cute graph theory algorithms.

The idea is that if you have the 0/1 adjacency matrix of a graph, and you take that matrix to the Nth power, then the (i,j) entry of the result tells how many paths of length N there are from vertex i to vertex j (here the length is measured in number of edges traversed) .

The only trick is instead of just counting the paths, to keep track of all the actual paths themselves. For this you have to slightly modify the matrix multiplication algorithm...

We modify the adjacency matrix so instead of being 0/1, the (i,j) entry of the matrix is a *list of paths* from i to j. Then in the matrix multiplication algorithm, instead of multiplying and adding entries, we instead concatenate pairs of paths together and union all of them (respectively). Here's what the code looks like:

```
use strict;

## just any old directed graph...

my $adj = [ [0,1,1,0,1],
            [1,0,0,0,1],
            [0,1,0,0,0],
            [1,1,1,0,0],
            [0,0,1,1,0] ];

my $dim = 4; ## size of the graph
my $N = 3;   ## length of paths (# of edges, not # of vertices).

## convert each entry in the adjacency matrix into a list of
```

| [Log in](#) | [Create a new user](#) | [The Monastery Gates](#)
 | [Super Search](#)
 | [Seekers of Perl Wisdom](#) | [Meditations](#)
 | [PerlMonks Discussion](#)
 | [Obfuscation](#) | [Reviews](#) | [Cool Uses For Perl](#) | [Perl News](#)
 | [Q&A](#) | [Tutorials](#)
 | [Poetry](#) | [Recent Threads](#) | [Newest Nodes](#) | [Donate](#)
 | [What's New](#)
[Need Help??](#)

Log In

Username:

Password:

☐ remember me

[What's my password?](#)

[Create A New User](#)

Node Status

[node history](#)

Node Type: note [id://522270]

[help](#)

Chatterbox

[[jellisii2](#)]: greetings

[How do I use this?](#) | [Other CB clients](#)

Other Users

Others musing on the Monastery:
(8)

[atcroft](#)

[afoken](#)

[Skeeve](#)

[Discipulus](#)

[monkey_boy](#)

[jellisii2](#)

[marioroy](#)

[ali0sha](#)

As of 2015-05-29 11:29 GMT

Sections

[Seekers of Perl Wisdom](#)

[Cool Uses for Perl](#)

[Meditations](#)

[PerlMonks Discussion](#)

[Categorized Q&A](#)

[Tutorials](#)

[Obfuscated Code](#)

[Perl Poetry](#)

[Perl News](#)

[about](#)

paths

```

for my $i (0 .. $dim) {
    for my $j (0 .. $dim) {
        $adj->[$i][$j] = $adj->[$i][$j] ? [$j] : [];
    }
}

## compute the $N-th power of the adjacency matrix with our
modified
## multiplication

my $result = $adj;
for (2 .. $N) {
    print_paths($result);
    print "=====\n";
    $result = matrix_mult($result, $adj);
}

print_paths($result);

## the i,j entry of the matrix is a list of all the paths from i
to j,
+ but
## without "i," at the beginning, so we must add it

sub print_paths {
    my $M = shift;
    my @paths;
    for my $i (0 .. $dim) {
        for my $j (0 .. $dim) {
            push @paths, map { "$i,$_" } @{$M->[$i][$j] };
        }
    }
    print map "$_\n", sort @paths;
}

## modified matrix multiplication. instead of multiplication, we
## combine paths from i->k and k->j to get paths from i->j (this
is wh
+y
## we include the endpoint in the path, but not the starting
point).
## then instead of addition, we union all these paths from i->j

sub matrix_mult {
    my ($A, $B) = @_;
    my $result;
    for my $i (0 .. $dim) {
        for my $j (0 .. $dim) {
            my @result;
            for my $k (0 .. $dim) {
                push @result, combine_paths( $A->[$i][$k], $B->[$k]
[$j] );
            }
            $result->[$i][$j] = \@result;
        }
    }
    $result;
}

## the sub to combine i->k paths and k->j paths into i->j paths
--
## we simply concatenate with a comma in between.

```

Information

[PerlMonks FAQ](#)
[Guide to the Monastery](#)
[What's New at PerlMonks](#)
[Voting/Experience System](#)
[Tutorials](#)
[Reviews](#)
[Library](#)
[Perl FAQs](#)
[Other Info Sources](#)

Find Nodes

[Nodes You Wrote](#)
[Super Search](#)
[List Nodes By Users](#)
[Newest Nodes](#)
[Recently Active Threads](#)
[Selected Best Nodes](#)
[Best Nodes](#)
[Worst Nodes](#)
[Saints in our Book](#)

Leftovers

[The St. Larry Wall Shrine](#)
[Buy PerlMonks Gear](#)
[Offering Plate](#)
[Awards](#)
[Random Node](#)
[Quests](#)
[Craft](#)
[Snippets](#)
[Code Catcombs](#)
[Editor Requests](#)
[blogs.perl.org](#)
[Perlsphere](#)
[Perl Ironman Blog](#)
[Perl Weekly](#)
[Perl.com](#)
[Perl 5 Wiki](#)
[Perl Jobs](#)
[Perl Mongers](#)
[Perl Directory](#)
[Perl documentation](#)
[MetaCPAN](#)
[CPAN](#)

Voting Booth

In my home, the TV remote control is ...

- ☐ Non-existent; I don't have TV
- ☐ Non-existent; my TV doesn't have/need a remote
- ☐ Lost
- ☐ Under my control; I watch alone
- ☐ Under the control of whoever shouts loudest
- ☐ Under the control of whoever gets it first
- ☐ One of many, and the TV has a mind of its own

```

sub combine_paths {
    my ($x, $y) = @_;
    my @result;

    for my $i (@$x) {
        for my $j (@$y) {
            push @result, "$i,$j";
        }
    }
    @result;
}

```

[\[download\]](#)

Output snippet:

```

0,1
0,2
...
=====
0,1,0
0,1,4
0,2,1
0,4,2
...
=====
...
0,4,2,1
0,4,3,0
0,4,3,1
0,4,3,2
1,0,1,0
1,0,1,4
1,0,2,1
...

```

[\[download\]](#)

blokhead

Comment on Re: find all paths of length n in a graph

Select or Download Code

[Re^2: find all paths of length n in a graph](#)

by [Anonymous Monk](#) on Sep 15, 2009 at 01:38 UTC

Sorry for restarting a closed thread. Can this be updated to find acyclic paths. i.e. I don't get these :- 1,0,1,0 1,0,1,4 1,0,2,1 Yes I can eliminate them after I've found them but that would mean a lot of work. Thank You, Himanshu

[\[reply\]](#)

[Re^3: find all paths of length n in a graph](#)

by [Anonymous Monk](#) on Sep 12, 2013 at 04:59 UTC

Depending on what you mean by acyclic paths, you might be able to. I would assume that if you can reach a point with a path of length 2, you don't want to count it again if you can reach it with a path of length 4. This situation might occur if these two points were a part of a path, and they could connect the short way--via two edges, or the long way--using 4 edges. In this case, just define an element wise

subtraction operation and subtract all the shorter paths. Aka, calculate $A^4 - A^3 - A^2 - A$. You might be able to make this faster by leaving markers in your matrix. For example, if two points have been connected using a shorter path, place a -1 into the matrix and force your multiplication algorithm to copy it when the matrix is multiplied. I.e., -1 always maps to -1.

[\[reply\]](#)

In Section [Seekers of Perl Wisdom](#)

PerlMonks graciously bestowed by [Tim Vroom](#).

PerlMonks somehow became entangled with [The Perl Foundation](#).

Speedy Servers and Bandwidth Generously Provided by [pair Networks](#)

Built with the [Perl programming language](#).